

Site Critique: thinktac.com

Performance/Technical

Opening the browser console on the homepage reveals a WhatsApp chat widget (whatsapp-widget.min.js) stuck in a broken initialization loop. Its `initWhatsAppWidget` function throws **TypeError: Cannot read properties of undefined (reading 'options')** on every attempt, but instead of failing once, it retries via `setTimeout` indefinitely. The stack trace shows it calling itself repeatedly (`initWhatsAppWidget` → `setTimeout` → `initWhatsAppWidget` → `setTimeout`...). This means the browser tab is doing unnecessary work in the background for as long as the page stays open, rather than encountering a one-time error.

Separately, the sticky header's search modal throws **Cannot read properties of null (reading 'close')** in `StickyHeader.closeSearchModal`, suggesting the modal element isn't present in the DOM when the header tries to reference it. This is likely caused by either a selector mismatch or a timing issue where the close handler executes before the modal has been initialized.

There is also a **SecurityError** generated by a script attempting to access a cross-origin iframe's document, which is blocked by the browser's Same-Origin Policy. This is most likely caused by a third-party embedded component (possibly the same WhatsApp widget) attempting to interact with an iframe without the required permissions.

UX/Product Gaps

None of these errors visibly break the page for a user, but they indicate that the WhatsApp widget itself likely never renders correctly, meaning the **"Chat with us"** entry point silently fails. A visitor trying to reach support via WhatsApp receives no feedback that the widget failed to load or initialize, resulting in a poor support experience despite the rest of the page appearing to function normally.

Concrete Suggestion

In `initWhatsAppWidget`, add a null check before accessing the `.options` property, and replace the unbounded polling mechanism with a capped retry count (for example, stop after five attempts) instead of retrying indefinitely with `setTimeout`.

This change converts a silent, resource-wasting failure into a clean, one-time console warning and forces whoever owns the widget to notice and fix the underlying issue instead of allowing it to fail invisibly on every page load. It also eliminates unnecessary background processing and reduces avoidable CPU usage while improving overall maintainability.